

AL-JUNAID TECH



Jupyter Notebook to PDF Converter

AI-powered tool that converts Jupyter notebooks to professionally formatted PDFs with proper markdown rendering.

**AUTHOR**

Muhammad Junaid

**VERSION**

1.0.0

**SOURCE**

LangChain_Tool_Docstrings.ipynb

**LANGUAGE**

Python

**CELLS**

8 total · 4 code · 4 markdown

**COMPANY**

AL-Junaid Tech

© 2026 AL-Junaid Tech. All rights reserved.

MARKDOWN

LangChain Tool Docstrings

Why Docstrings Matter in LangChain Agents

When you pass a Python function as a **tool** to `create_agent()` in LangChain, LangChain reads that function's **docstring** and tells the LLM what the tool does. Without a docstring, the LLM cannot understand the tool — and the agent will not even be created.

Step 1 — Setup

-  Load `MODEL_NAME` and `TEMPERATURE` from `.env`
-  Initialize the `ChatOllama` LLM
-  The same LLM will be used for both agents (with and without docstring)

CODE

[1]

```
from datetime import datetime
from zoneinfo import ZoneInfo
from langchain_core.messages import HumanMessage
from langchain.agents import create_agent
from langchain_ollama import ChatOllama
import os
from dotenv import load_dotenv

# --- Load Environment Variables ---
load_dotenv()
MODEL_NAME = os.getenv("MODEL_NAME", "minimax-m2.5:cloud")
TEMPERATURE = float(os.getenv("TEMPERATURE", "0.7"))

# Initialize LLM
llm = ChatOllama(model=MODEL_NAME, temperature=TEMPERATURE)
print(f"LLM initialized: {llm.model}")
```

STDOUT

```
LLM initialized: minimax-m2.5:cloud
```

MARKDOWN

Step 2 — Agent WITH Docstring

-  Both `get_weather_with_doc` and `get_time` have a **docstring**
-  The LLM reads the docstring and knows when and how to call the tool
-  The agent is created successfully and returns the correct output

CODE

[2]

```
# -----
# WITH docstring
# -----
def get_weather_with_doc(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

def get_time(city: str) -> str:
    """Get current time for a given city."""
    tz = ZoneInfo("Asia/Karachi")
    now = datetime.now(tz).strftime("%I:%M %p")
    return f"The current time in {city} is {now}"

agent_with_doc = create_agent(
    llm,
    tools=[get_weather_with_doc, get_time],
    system_prompt="You are a helpful assistant with weather/time tools."
)

result_with_doc = agent_with_doc.invoke({
    "messages": [HumanMessage(content="Weather and time in Islamabad?")]
})

print("\n--- Output WITH docstring ---")
print(result_with_doc["messages"][-1].content)
```

STDOUT

```
--- Output WITH docstring ---
Here's the current weather and time for Islamabad:

Weather: It's always sunny in Islamabad! ☀️

Time: The current time in Islamabad is 04:53 AM.

Is there anything else you'd like to know?
```

MARKDOWN

✗ Step 3 — Agent WITHOUT Docstring

- ☹️ `get_weather_without_doc` has a **missing docstring**
- ⚠️ LangChain cannot send a tool description to the LLM
- 💥 `create_agent()` immediately raises a `ValueError`

CODE

[3]

```
# -----
# WITHOUT docstring
# -----
def get_weather_without_doc(city: str) -> str:
    return f"It's always sunny in {city}!"

try:
    agent_without_doc = create_agent(
        llm,
        tools=[get_weather_without_doc, get_time],
        system_prompt="You are a helpful assistant with weather/time tools."
    )
    result_without_doc = agent_without_doc.invoke({
        "messages": [HumanMessage(content="Weather and time in Islamabad?")]
    })
    print("\n--- Output WITHOUT docstring ---")
    print(result_without_doc["messages"][-1].content)
except Exception as e:
    print("\n--- Error WITHOUT docstring ---")
    print(e)
```

STDOUT

```
--- Error WITHOUT docstring ---
Function must have a docstring if description not provided.
```

MARKDOWN

 Step 4 — Explanation

- 💡 With a docstring, the LLM can use the tool correctly
- 🚫 Without a docstring, the agent is not created at all (`ValueError`)
- 🔑 A docstring is the **required description** for LangChain tools — not just a comment

 Bottom Line

Always write a docstring in every tool function — otherwise the agent will not build.

CODE

[4]

```
# -----  
# Explanation  
# -----  
explanation = """  
Explanation:  
1. WITH docstring: The LLM knows what the 'get_weather_with_doc' function does  
   and can call it correctly when needed.  
2. WITHOUT docstring: LangChain tries to generate a tool description automatically  
   from the docstring. Since it is missing, the agent cannot use the function,  
   resulting in a ValueError or the agent ignoring the tool.  
3. Docstrings are critical in LangChain for passing descriptions of tools to the LLM.  
"""  
print(explanation)
```

STDOUT

```
Explanation:  
1. WITH docstring: The LLM knows what the 'get_weather_with_doc' function does  
   and can call it correctly when needed.  
2. WITHOUT docstring: LangChain tries to generate a tool description automatically  
   from the docstring. Since it is missing, the agent cannot use the function,  
   resulting in a ValueError or the agent ignoring the tool.  
3. Docstrings are critical in LangChain for passing descriptions of tools to the  
   LLM.
```